

Real-Time Chat Application using WebSockets and Serverless

Project: Real-Time Chat Application using WebSockets and Serverless

Services: API Gateway (WebSocket API), AWS Lambda, DynamoDB, Amazon Cognito

Description: Develop a scalable real-time chat application using AWS serverless services and WebSocket communication. The application enables multiple authenticated users to exchange messages instantly using persistent WebSocket connections. AWS Lambda processes user connections and messages, DynamoDB stores active connection IDs and chat data, API Gateway manages WebSocket communication, while Amazon Cognito provides secure user signup, login, authentication, and identity federation.

Skills: WebSocket management, Serverless computing, Identity federation, User authentication, State handling, Real-time communication, AWS Lambda scripting.

Definition: The Real-Time Chat Application using WebSockets and Serverless Architecture is designed to provide secure and scalable instant messaging capabilities between multiple authenticated users over persistent WebSocket connections.

The system uses Amazon API Gateway WebSocket APIs for real-time bidirectional communication, AWS Lambda for backend processing, DynamoDB for storing active client connections and chat-related state information, and Amazon Cognito for secure user signup, login, authentication, and identity management.

Amazon Cognito enables secure user registration and authentication before accessing the chat platform, ensuring only authenticated users can connect to the WebSocket API and exchange messages. The architecture is fully serverless, scalable, event-driven, and capable of handling multiple simultaneous client connections with minimal operational overhead.

Scope of the Project:

The scope of this project is to design and implement a scalable real-time messaging platform using AWS serverless services and WebSocket APIs. The project focuses on enabling secure, authenticated, real-time communication between connected users through persistent WebSocket connections.

This project covers:

- **Real-Time WebSocket Communication:** Enables bidirectional communication between users and the server using API Gateway WebSocket APIs.
- **User Authentication and Authorization:** Implements secure user signup and login using Amazon Cognito.
- **User Connection Management:** Stores and manages active WebSocket connection IDs using DynamoDB.
- **Message Broadcasting:** Broadcasts chat messages to all connected users in real time using AWS Lambda.
- **Fully Serverless Architecture:** Uses AWS managed services without provisioning or managing servers.
- **Scalable Event-Driven Design:** Automatically scales based on active users and WebSocket traffic.

Methodology:

1.API Gateway + Lambda + DynamoDB + Cognito – Best Method

- Uses all required AWS services:API Gateway (WebSocket API), Lambda, DynamoDB, and Cognito.
- Enables real-time bidirectional communication using persistent WebSocket connections.
- Provides scalable serverless architecture without managing servers.
- Supports instant message broadcasting to multiple authenticated users.
- Uses DynamoDB for storing active WebSocket connection IDs and chat state information.
- Uses Amazon Cognito for secure user signup, login, authentication, and identity federation.
- Fully automated and event-driven architecture suitable for organizational deployment.
- Cost-effective and Free-tier friendly for small workloads.

2.EC2 Socket Server + Database – Alternative Method

- Uses EC2 instances to host custom WebSocket servers.
- Requires manual configuration, scaling, patching, monitoring, and security management.
- Uses traditional socket programming frameworks.
- Requires continuously running EC2 instances.
- Higher operational cost and maintenance effort compared to serverless architecture.
- Suitable for legacy applications requiring dedicated server infrastructure.

AWS Services used in the Project:

1.Amazon API Gateway (WebSocket API)

Definition: Amazon API Gateway is a managed AWS service that allows developers to create, publish, monitor, and secure APIs at any scale.

Purpose: Provides WebSocket APIs for real-time bidirectional communication.

Role:

- Maintains persistent WebSocket client connections.
- Handles connect and disconnect events.
- Routes incoming messages to Lambda functions.
- Enables real-time communication between users.

2.AWS Lambda

Definition: AWS Lambda is a serverless compute service that runs code without provisioning or managing servers.

Purpose: Processes WebSocket events and chat messages.

Role:

- Handles WebSocket connect events.
- Stores active connection IDs in DynamoDB.
- Removes disconnected users.
- Broadcasts messages to connected clients.

Lambda Functions Used:

- connectHandler
- disconnectHandler
- sendMessageHandler

3. Amazon DynamoDB

Definition: Amazon DynamoDB is a fully managed NoSQL database service providing fast and flexible key-value storage.

Purpose: Stores active WebSocket connection IDs and chat-related information.

Role:

- Maintains active user connection records.
- Supports real-time message broadcasting.
- Provides scalable state management for connected clients.

DynamoDB Tables Used:

- Connections Table
- Messages Table

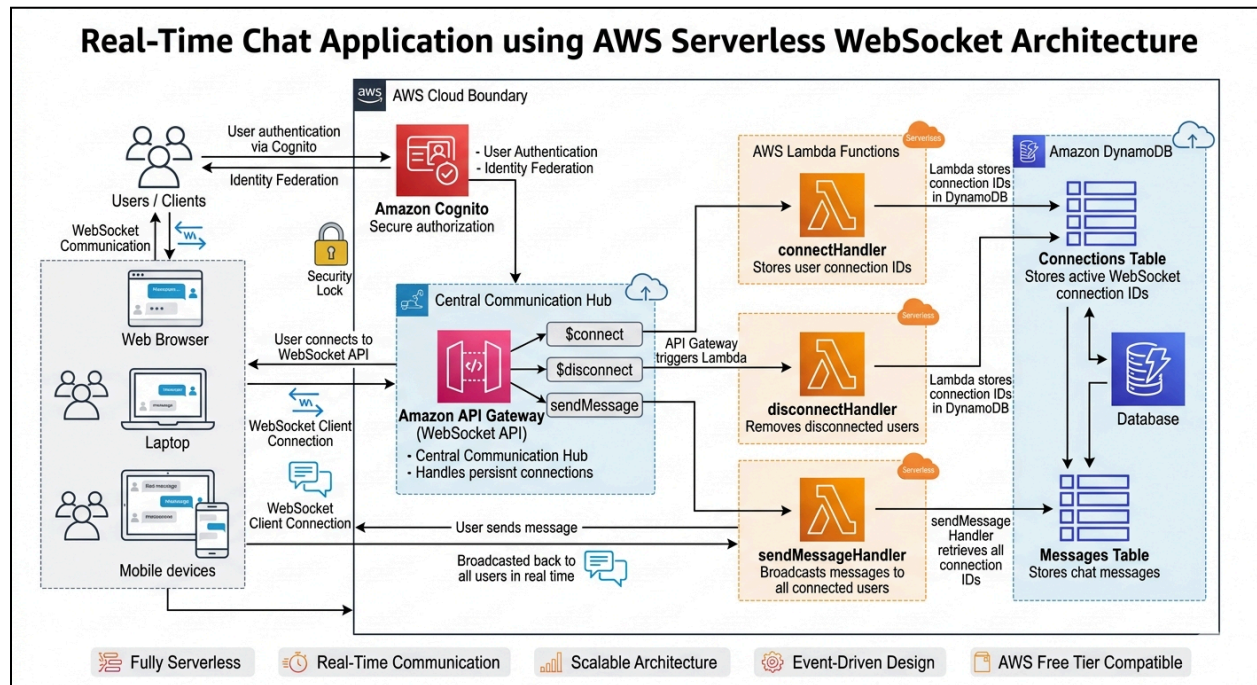
4. Amazon Cognito

Definition: Amazon Cognito is an authentication and user management service that provides secure signup, login, and access control for applications.

Purpose: Provides secure user authentication and identity federation.

Role:

- Enables user signup and login functionality.
- Authenticates users before accessing the chat application.
- Manages user identities securely.
- Provides authorization and access management.

Architecture Diagram:**Implementation Steps to Do the Project: Real-Time Chat Application using WebSockets and Serverless****Step1: Create DynamoDB Tables**

Go to AWS Console → DynamoDB → Create Table.

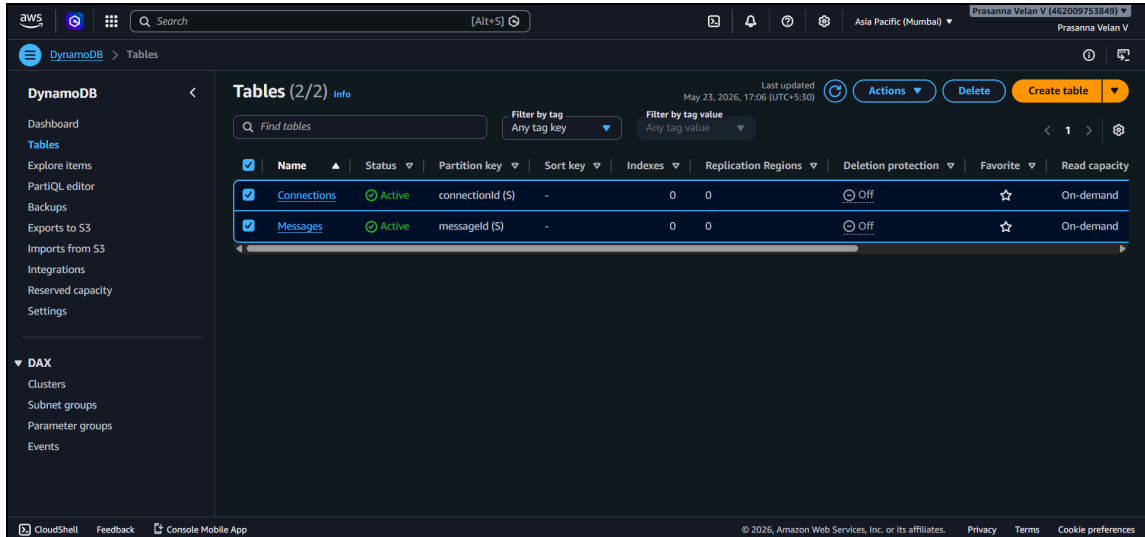
Create Table 1:

- Table Name: Connections
- Partition Key: connectionId (String)

Create Table 2:

- Table Name: Messages
- Partition Key: messageId (String)

Real-Time Chat Application using WebSockets and Serverless



Step 2: Create Cognito User Pool

Go to AWS Console → Cognito → Create User Pool.

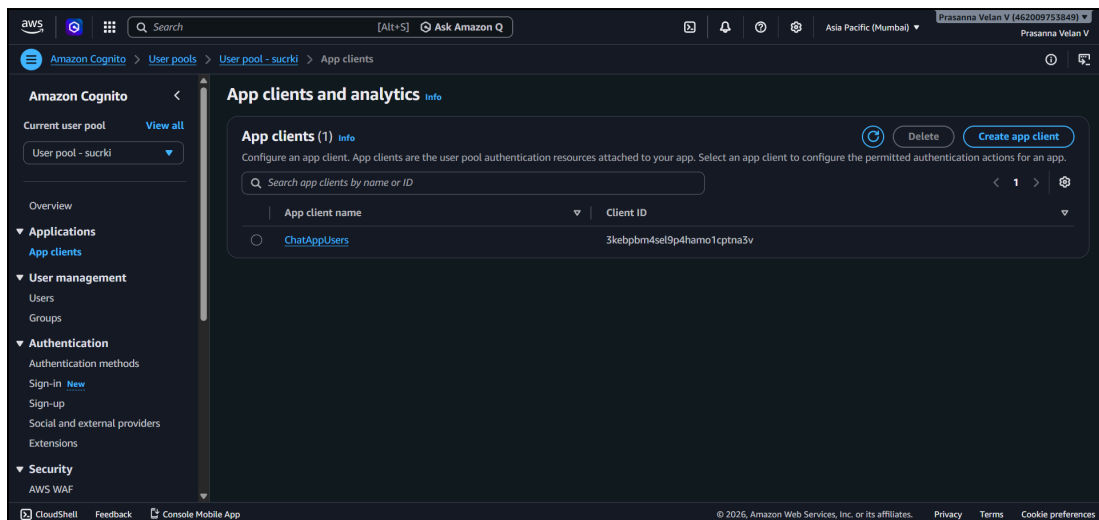
Configure:

- User Pool Name: ChatAppUsers
- Sign-in Options: Email
- Password Policy: Default

Create App Client.

Enable: Username and Password Authentication

Create test users for login and signup verification.



Step 3: Create IAM Role for Lambda

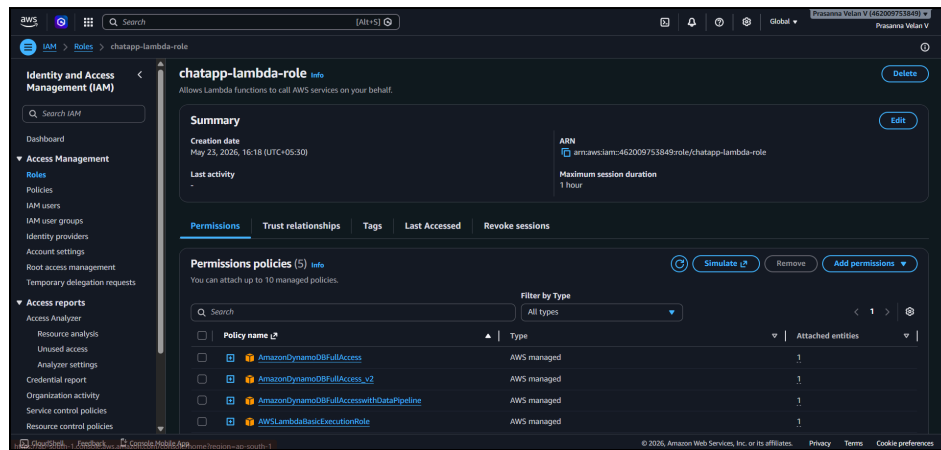
Go to IAM → Roles → Create Role → Lambda.

Attach Policies:

- AWSLambdaBasicExecutionRole
- AmazonDynamoDBFullAccess

Add Inline Policy: execute-api:ManageConnections

Name the role: Chatapp-lambda-role



Step 4: Create Lambda Functions

Go to Lambda → Create Function → Author from Scratch.

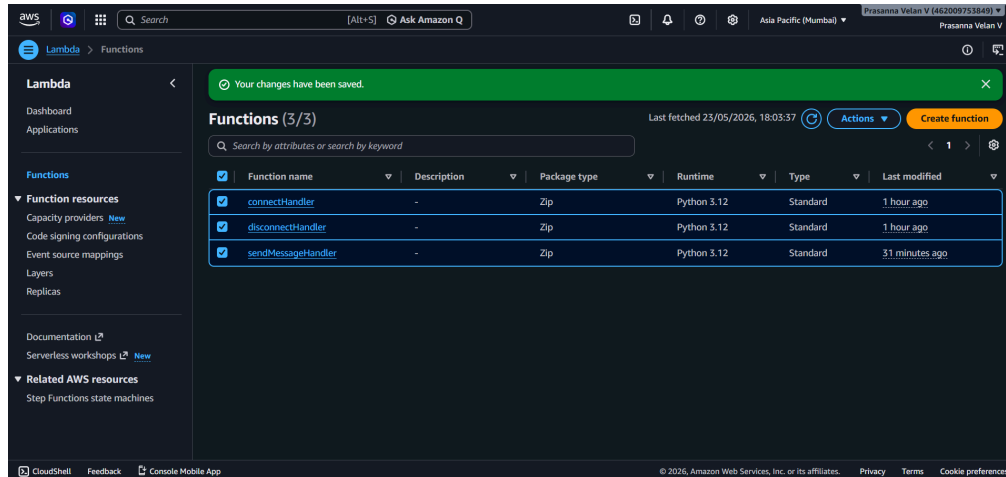
Create Functions:

1. connectHandler
2. disconnectHandler
3. sendMessageHandler

Runtime: Python 3.12

Execution Role: Use Existing Role:
chatapp-lambda-role

Real-Time Chat Application using WebSockets and Serverless

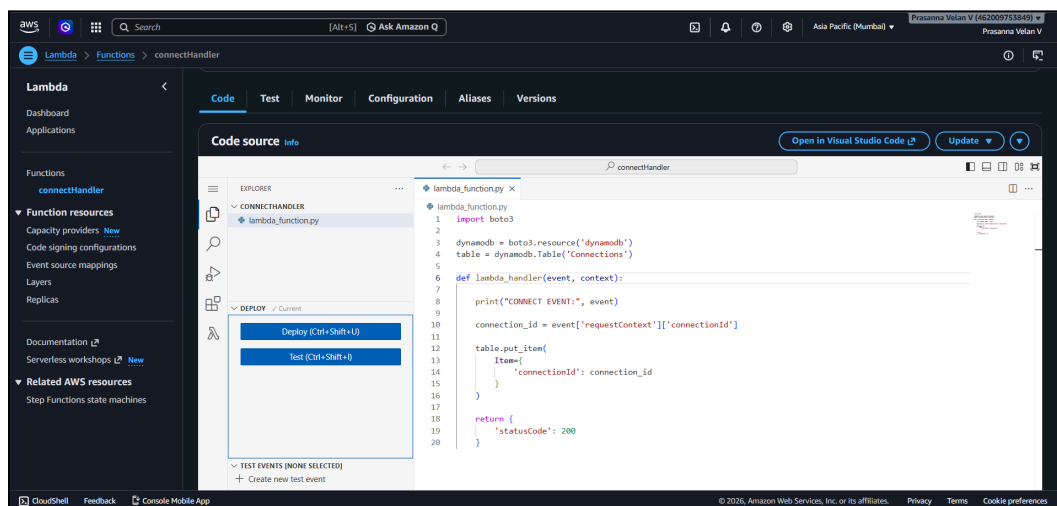


Step 5: Add Lambda Function Code and Test

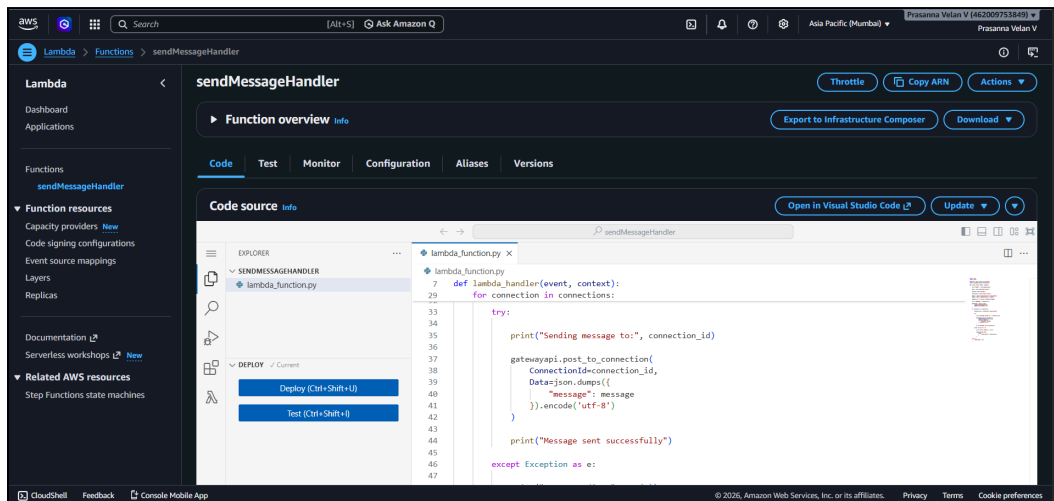
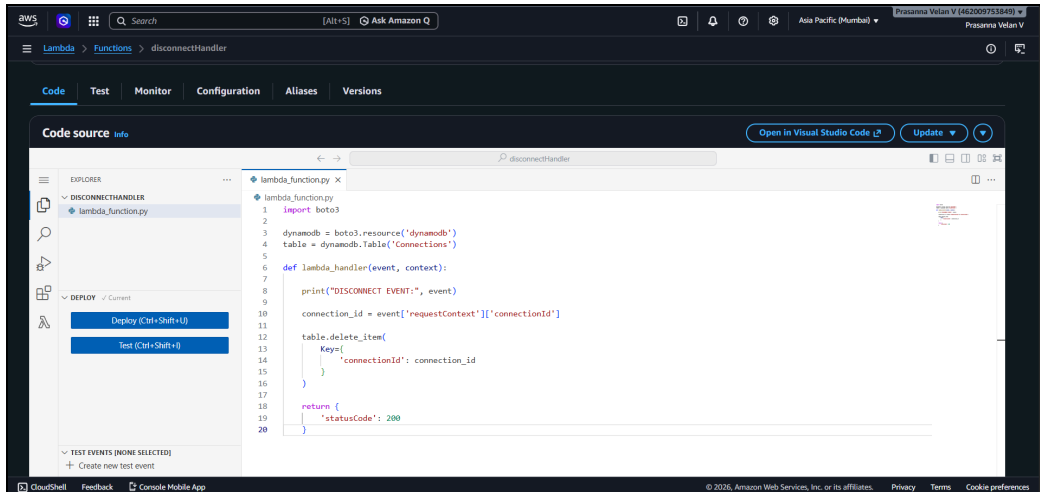
Add Python code for: WebSocket connection handling, Message broadcasting, Connection state management

Click Deploy after adding code.

Verify CloudWatch logs for successful execution.



Real-Time Chat Application using WebSockets and Serverless



Step 6: Create WebSocket API in API Gateway

Go to API Gateway → Create API.

Select: WebSocket API

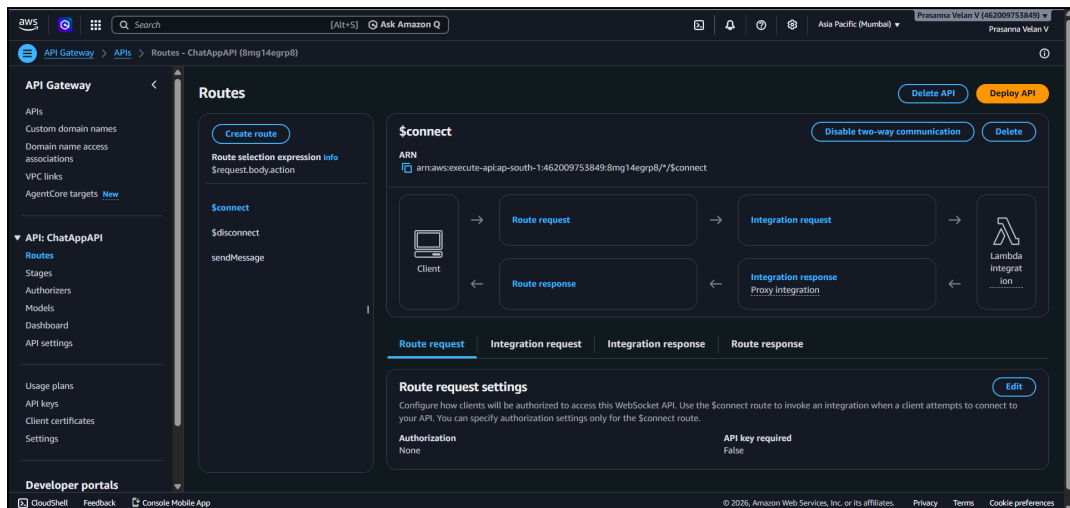
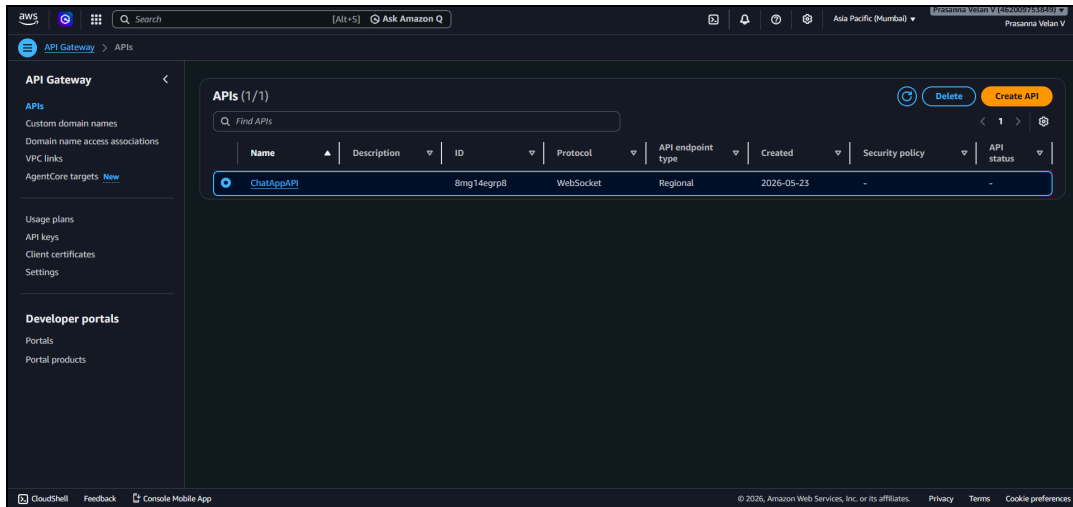
Configure: Route Selection Expression:
\$request.body.action

Create Routes: \$connect , \$disconnect ,sendMessage

Attach Lambda Integrations: connectHandler ,disconnectHandler ,sendMessageHandler

Enable: Lambda Proxy Integration

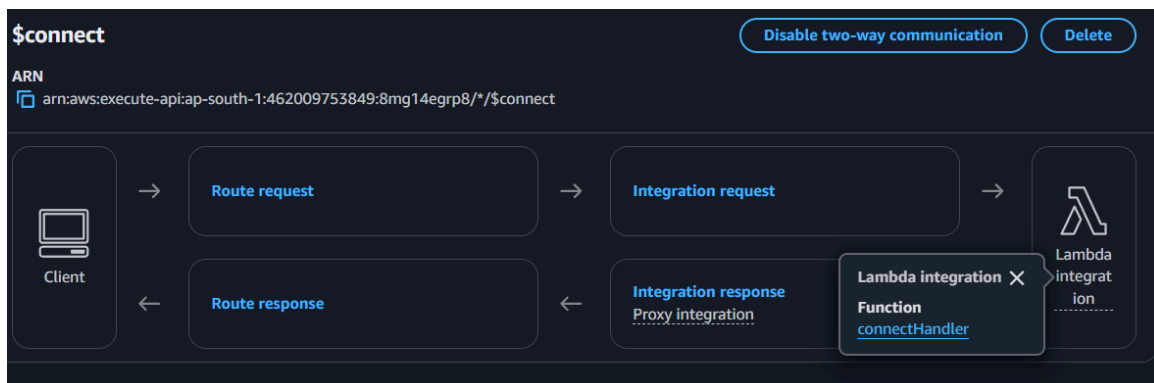
Real-Time Chat Application using WebSockets and Serverless



Step 7: Configure Lambda Permissions

Add API Gateway invoke permissions to all Lambda functions.

Add execute-api:ManageConnections permission to Lambda execution role.



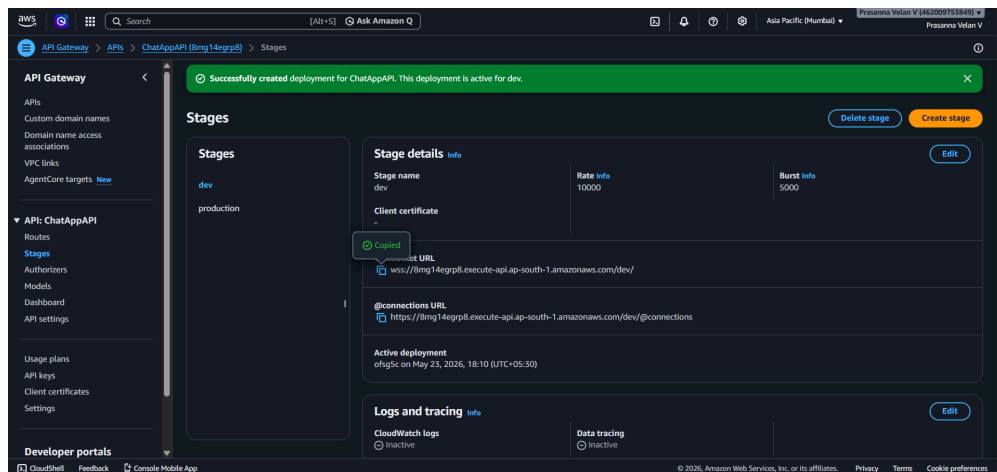
Step 8: Deploy WebSocket API

Create Stage: dev

Deploy API.

Copy WebSocket Endpoint URL:

wss://8mg14egrp8.execute-api.ap-south-1.amazonaws.com/dev/

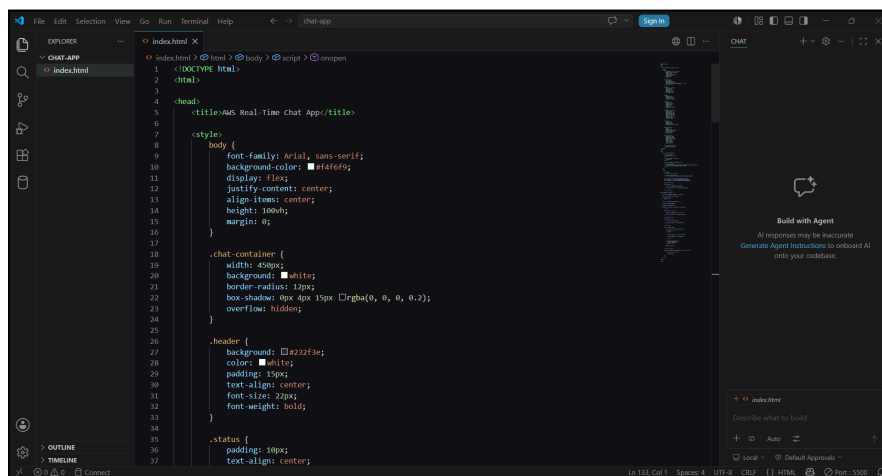


Step 9: Create Frontend Chat Application

Create index.html using: HTML ,CSS

Implement:

- Cognito Login and Signup
- WebSocket connection
- Message sending
- Real-time message receiving
- Connection status handling

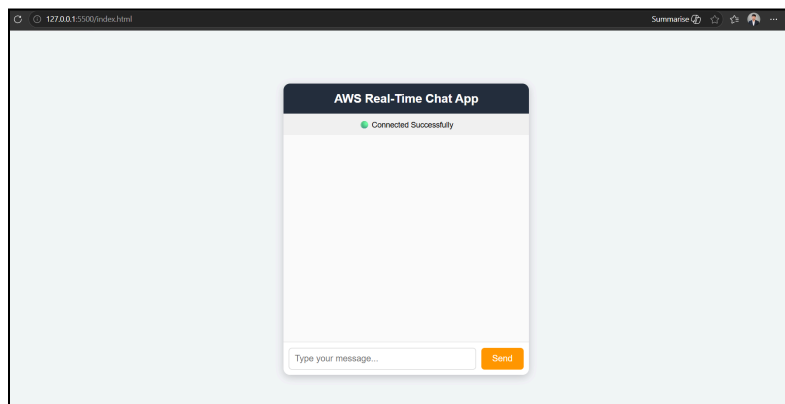
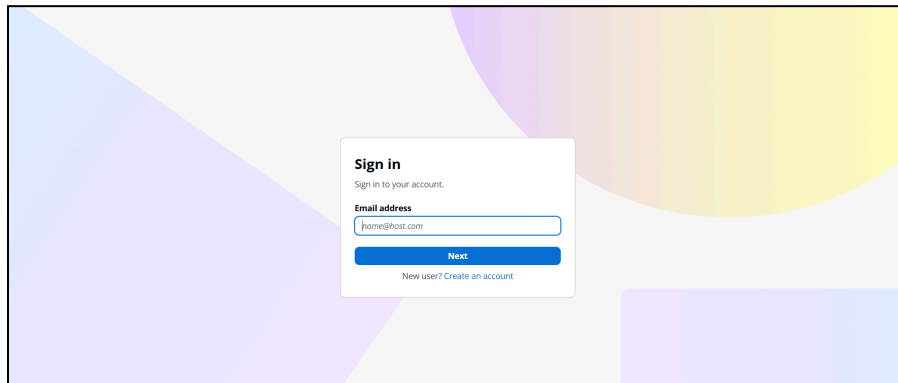
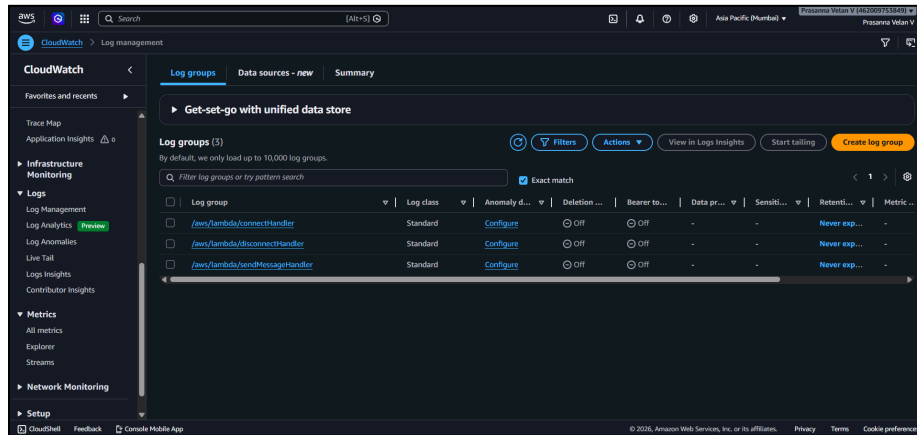


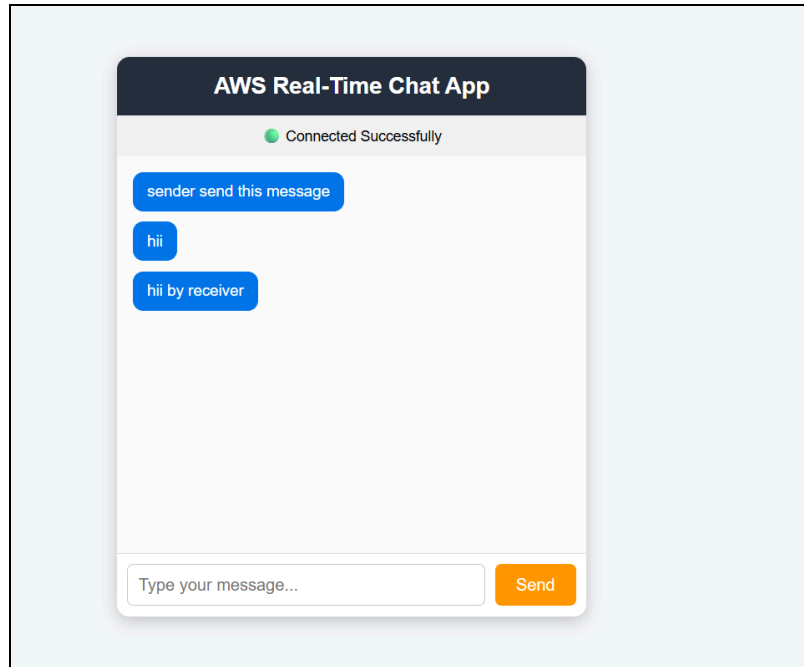
Step 10: Test the Application

Open application in multiple browser tabs.

Verify:

- User signup using Cognito
- User login authentication
- Successful WebSocket connection
- Real-time message broadcasting
- Multi-user communication





Troubleshooting:

Issue 1: WebSocket Connection Failed

Cause: API Gateway integration or permissions were incorrectly configured.

Solution:

- Enable Lambda Proxy Integration.
- Verify WebSocket endpoint URL.
- Redeploy API Gateway stage.
- Add API Gateway invoke permissions.

Issue 2: AccessDeniedException for execute-api:ManageConnections

Cause: Lambda execution role lacked WebSocket management permissions.

Solution:

- Add inline IAM policy:
execute-api:ManageConnections
- Attach policy to Lambda execution role.

Issue 3: Lambda Handler Error

Cause: Incorrect Lambda handler configuration or missing requestContext.

Solution:

- Verify Lambda handler:
`lambda_function.lambda_handler`
- Ensure requests are triggered from WebSocket API instead of manual Lambda test events.

Conclusion:

The Real-Time Chat Application using WebSockets and Serverless Architecture provides a scalable, secure, and event-driven messaging platform using AWS cloud services. Using API Gateway WebSocket APIs, AWS Lambda, DynamoDB, and Amazon Cognito, the system enables authenticated users to exchange messages instantly through persistent WebSocket connections while eliminating the need for server management.

Amazon Cognito provides secure signup, login, authentication, and identity management, ensuring only authenticated users can access the platform. The serverless architecture automatically scales based on demand and significantly reduces operational overhead and infrastructure costs.

This project demonstrates practical implementation of:

- Real-time WebSocket communication
- Serverless computing
- User authentication and identity federation
- State management
- Cloud-native application development using AWS services

The solution is suitable for educational projects, organizational deployment, and real-time communication applications while remaining compatible with AWS Free Tier for small-scale workloads.